LangChain

Connecting RAG Components

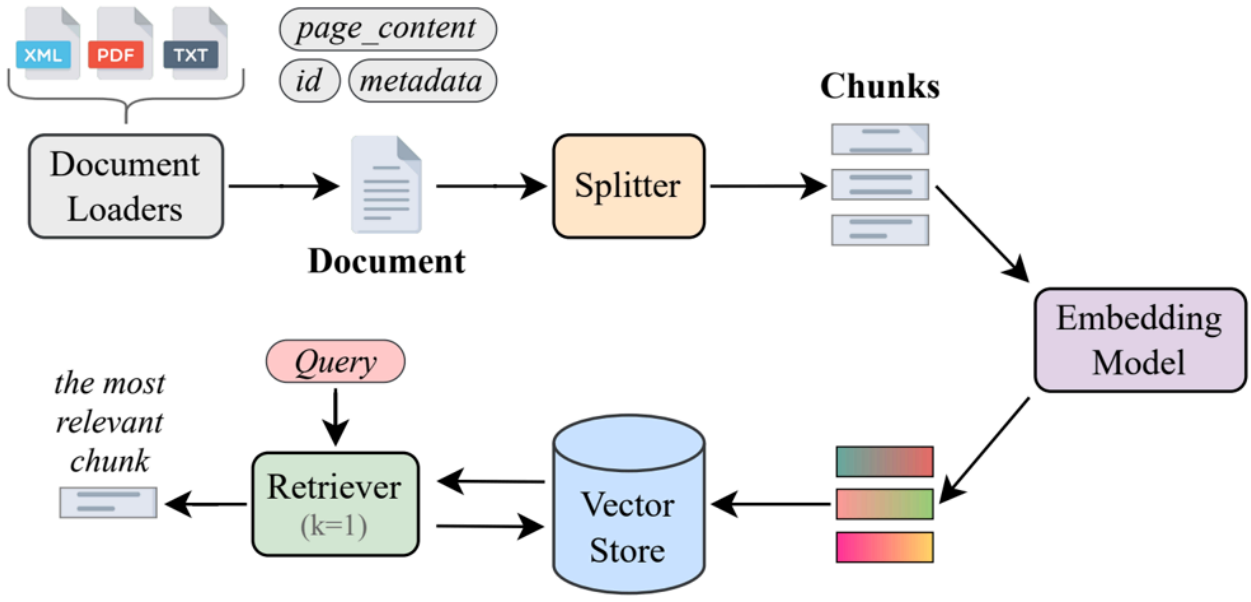
Definition:

LangChain is an open-source framework used to develop applications powered by Large Language Models (LLMs). It connects different components of an LLM application into a single workflow.

It helps developers integrate:

- LLMs (GPT, Gemini, Llama, etc.)
- Document loaders
- Text splitters
- Embedding models
- Vector databases
- Retrievers
- Prompt templates

Working of LangChain in RAG:



The typical workflow is:

1. Document Loading: Load data from PDFs, text files, websites or CSV files.
2. Splitting: Divide large documents into smaller chunks.
3. Embedding: Convert text chunks into numerical vectors.
4. Storage: Store embeddings in a vector database.

5. Retrieval: Retrieve relevant chunks based on the user's query.
6. Generation: Send retrieved information to the LLM to generate an answer.

Important components:

Component	Purpose
Document Loaders	Load documents
Text Splitters	Divide documents into chunks
Embeddings	Convert text into vectors
Vector Stores	Store embeddings
Retrievers	Find relevant information
Chains	Connect multiple operations
Prompt Templates	Structure prompts for LLMs

Example use case:

Building a chatbot that answers questions from company policy PDFs.

If a user asks, "What is the company's leave policy?", LangChain retrieves relevant information from the PDF and sends it to the LLM to generate an answer.

Advantages:

- Flexible and modular.
- Supports multiple LLM providers.
- Integrates with different vector databases.

- Useful for chatbots, agents and RAG applications.

LlamaIndex

Document Indexing and Retrieval

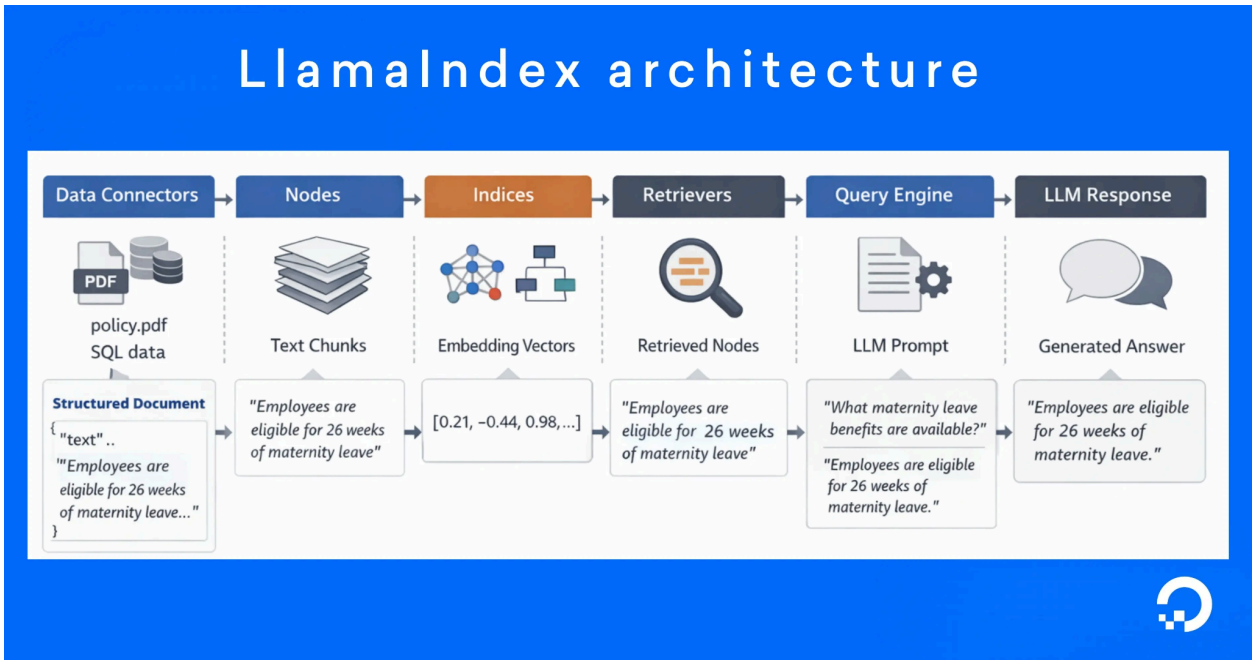
Definition:

LlamaIndex is a framework designed to connect Large Language Models with external data.

It focuses primarily on making private or external data accessible to LLMs through efficient indexing and retrieval.

It is particularly useful when working with large collections of documents.

Working of LlamaIndex in RAG:



The typical workflow is:

1. Document Loading: Import documents from different sources.
2. Node Creation: Divide documents into smaller units called nodes.
3. Indexing: Organize nodes into an index for efficient searching.
4. Embedding: Convert nodes into vector representations.
5. Retrieval: Search the index for relevant nodes.
6. Response Generation: Pass retrieved information to an LLM to generate an answer.

Important concepts:

Concept	Purpose
Documents	Original data sources
Nodes	Smaller pieces of documents
Index	Organizes information for retrieval
Vector Store Index	Stores and searches vector embeddings
Retriever	Retrieves relevant nodes
Query Engine	Combines retrieval and response generation
Response Synthesizer	Generates responses using retrieved information

What are Nodes?

Nodes are smaller units of information created by splitting documents.

For example:

Original document: 1 PDF containing 100 pages.

After splitting:

- Node 1: Pages 1–5
- Node 2: Pages 6–10
- Node 3: Pages 11–15

Each node can be embedded and indexed for retrieval.

Example use case:

Building an AI assistant that answers questions from thousands of company documents.

Advantages:

- Excellent for document-heavy RAG applications.
- Provides different indexing strategies.
- Supports structured and unstructured data.
- Makes document retrieval easier.
- Supports multiple data connectors and vector databases.

Haystack

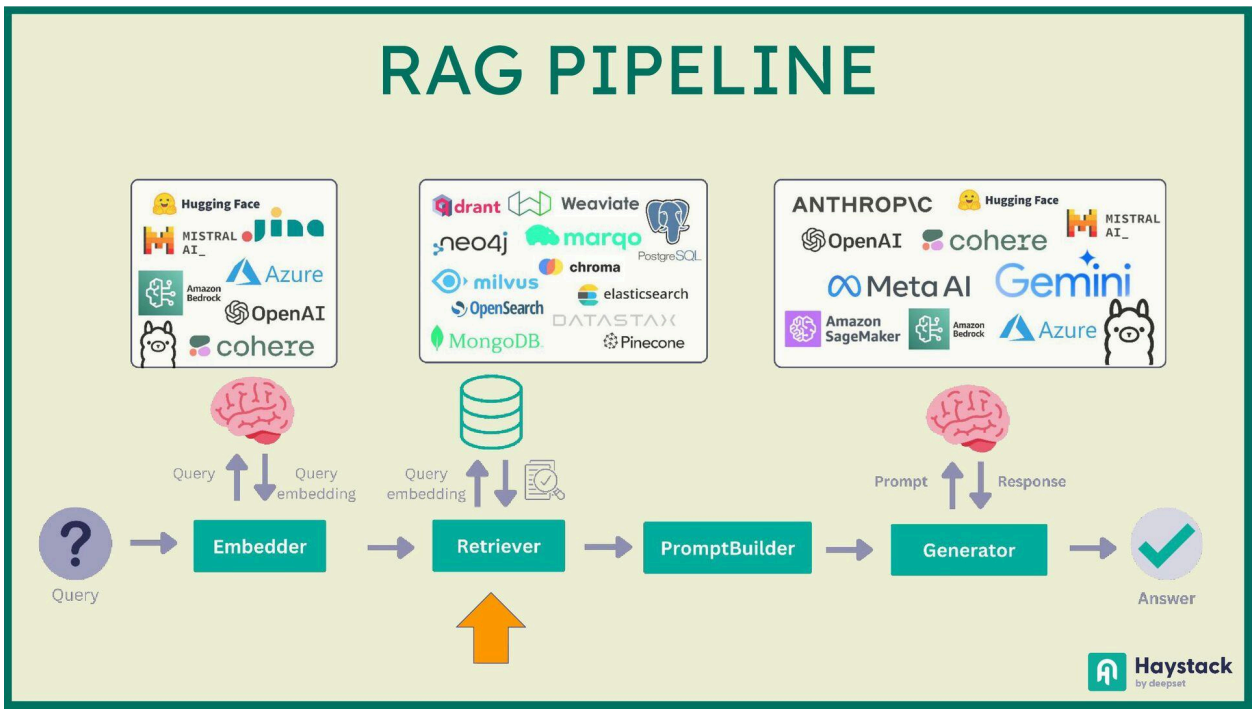
Building RAG Pipelines

Definition:

Haystack is an open-source framework used to build AI applications, especially search systems, question-answering systems and RAG pipelines.

It allows developers to create modular pipelines by connecting individual components.

Working of Haystack in RAG:



The typical workflow is:

1. Document Store: Stores documents and their embeddings.

2. Document Embedder: Converts documents into vector representations.
3. Retriever: Searches for relevant documents.
4. Prompt Builder: Creates a prompt using the retrieved documents and user query.
5. Generator: Uses an LLM to generate the final answer.
6. Results: Returns the generated response to the user.

Important components:

Component	**Purpose**
Document Store	Stores documents
Embedder	Converts documents into vectors
Retriever	Finds relevant documents
Prompt Builder	Creates prompts
Generator	Generates answers using an LLM
Pipeline	Connects different components

Example use case:

Building a customer-support chatbot that retrieves information from product manuals and generates answers to customer questions.

Advantages:

- Modular pipeline architecture.
- Easy integration of different components.
- Suitable for production-level RAG applications.
- Supports document search and question answering.
- Provides flexibility in designing AI workflows.

Comparison: LangChain vs LlamaIndex vs Haystack

This is an important comparison to understand for interviews and technical assessments.

Feature	LangChain	LlamaIndex	Haystack
Main purpose	Connecting LLM components	Document indexing and retrieval	Building AI pipelines
Main focus	LLM application development	Data ingestion and retrieval	Search and RAG pipelines
Core concept	Chains and components	Documents, nodes and indexes	Components and pipelines
Document handling	Supported	Strong focus	Supported
Retrieval	Supported	Strong focus	Strong focus
LLM integration	Yes	Yes	Yes
Best suited for	Complex LLM applications	Document-based RAG	Modular search and RAG systems